

Clasificador KNN con características mejoradas

Objetivo:

Se entrenará un modelo capaz de distinguir entre **dos objetos reales** (por ejemplo, una taza y una botella) capturados con la webcam. Para lograr una buena clasificación, se extraen **características significativas** de cada imagen.

¿Qué es KNeighborsClassifier?

KNeighborsClassifier es una **clase de Python** que forma parte del módulo `sklearn.neighbors` de la biblioteca **Scikit-learn**.

Esta clase permite crear un modelo de **Machine Learning supervisado** basado en el algoritmo **K-Nearest Neighbors (KNN)**, o en español, **K vecinos más cercanos**.

¿Cómo funciona el algoritmo KNN?

Es un algoritmo muy intuitivo:

1. **Memoriza los datos de entrenamiento.**
No aprende reglas, solo guarda los datos con sus etiquetas.
2. Cuando llega un dato nuevo para clasificar:
 - Calcula la **distancia** entre ese dato y todos los datos de entrenamiento.
 - Encuentra los **K ejemplos más cercanos** (por eso "K vecinos").
 - **Vota** entre esos K vecinos para decidir a qué clase pertenece el nuevo dato.

Ejemplo real:

Supón que tienes una base con datos de frutas: manzanas y peras.

- Cada fruta tiene características como **peso y color**.
- Si llega una fruta nueva que pesa 150g y es verde claro, KNN buscará las frutas más parecidas en la base y, si por ejemplo 2 de 3 frutas cercanas son peras, clasificará la nueva fruta como **"pera"**.

¿Cuándo usar KNeighborsClassifier?

- Cuando los datos no tienen muchas dimensiones (es decir, no demasiadas características).
- Cuando quieres un modelo fácil de entender y de implementar.
- Para clasificación de imágenes simples, colores, formas, audio básico, etc.

PASO A PASO

1. Importar las librerías

```
from sklearn.neighbors import KNeighborsClassifier
```

¿Qué hacen?

- cv2: para manejar imágenes y video con OpenCV.
- numpy: para manejar arreglos numéricos y vectores.
- KNeighborsClassifier: el clasificador que aprenderá a distinguir los objetos.

2. Definir la función `extraer_caracteristicas_avanzadas()`

```
def extraer_caracteristicas_avanzadas(imagen):  
    mini = cv2.resize(imagen, (100, 100))  
    gray = cv2.cvtColor(mini, cv2.COLOR_BGR2GRAY)  
    # Color  
    color_mean = mini.mean(axis=(0, 1))  
    color_std = mini.std(axis=(0, 1))  
    median_color = np.median(mini.reshape(-1, 3), axis=0)  
    # Textura  
    lap_var = cv2.Laplacian(gray, cv2.CV_64F).var()  
    edge_count = np.count_nonzero(cv2.Canny(gray, 100, 200))  
    # Forma  
    _, thresh = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)  
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE)  
    area = cv2.contourArea(max(contours, key=cv2.contourArea)) if contours else 0  
  
    return np.concatenate([  
        color_mean, color_std, median_color,  
        [lap_var, edge_count, area]  
    ])
```

¿Qué hace?

- Reduce la imagen para simplificar el análisis.
- Convierte la imagen a **escala de grises**.
- Extrae 3 tipos de características:

- **Color:** promedio,

¿Qué hace `.mean(axis=(0, 1))`?

Calcular el promedio en las dimensiones 0 y 1, o sea, promediar todos los píxeles, pero manteniendo los 3 canales de color separados.

- desviación estándar,
- mediana (cada uno con 3 valores RGB).

¿Qué hace `reshape(-1, 3)`?

Convierte esa imagen de forma (100, 100, 3) en una **lista plana de píxeles RGB**, uno por fila.

`mini.reshape(-1, 3) → (10000, 3)`

Cada **fila** representa un píxel.

Cada **columna** representa un canal de color: R, G, B.

¿Y por qué -1?

En NumPy, el -1 significa:

"calcula automáticamente cuántas filas debe haber".

Como la imagen tenía $100 \times 100 = 10,000$ **píxeles**, `reshape(-1, 3)` genera una matriz de 10,000 filas $\times$ 3 columnas, sin que tengas que calcular el 10000 a mano.

`axis=0` significa: calcular la mediana **por columna** (es decir, por canal R, G, B).

- **Textura:**

- `lap_var`: mide la nitidez.

`cv2.Laplacian(gray, cv2.CV_64F)`

Aplica el filtro de Laplaciano a la imagen `gray`.

`gray` debe ser una imagen en escala de grises (una matriz 2D).

¿Qué es el filtro de Laplaciano?

Es un operador que detecta bordes. Busca zonas de cambio brusco de intensidad (es decir, dónde termina un objeto y empieza otro).

Detecta bordes en todas las direcciones (horizontal, vertical y diagonal).

cv2.CV_64F indica que el resultado se guarda en punto flotante de 64 bits (más precisión y permite valores negativos).

- **edge_count:** cantidad de bordes (Canny).
- **Forma:**
 - area: área del contorno más grande.

El vector final tiene $3+3+3+1+1+1 = 12$ **características** por imagen.

Explicación de: cv2.threshold(...)

Esta función aplica **umbralización** a una imagen. Sirve para **separar objetos del fondo** según la intensidad de los píxeles

Argumentos:

gray: Es la imagen en escala de grises (matriz de valores de 0 a 255).

Cada píxel representa una intensidad: 0 = negro, 255 = blanco.

100 (umbral): Es el valor de corte.

Si un píxel es mayor o igual que 100, se convierte en blanco (255).

Si es menor, se convierte en negro (0).

255: Es el valor que se asigna a los píxeles que cumplen la condición.

Es decir, los que sean ≥ 100 se volverán blancos (255).

cv2.THRESH_BINARY: Modo de umbralización binaria:

Píxeles $\geq$ umbral $\rightarrow$ 255 (blanco)

Píxeles $<$ umbral $\rightarrow$ 0 (negro)

¿Y para qué sirve esta imagen binaria?

- Para **detectar contornos**
- Para **segmentar objetos**
- Para **simplificar** la imagen antes de extraer características

Argumentos usados en la función:

1. **gray**
 - Es la imagen en **escala de grises** (matriz de valores de 0 a 255).
 - Cada píxel representa una intensidad: 0 = negro, 255 = blanco.
2. **100** (umbral)
 - Es el valor de corte.

- Si un píxel es **mayor o igual que 100**, se convierte en blanco (255).
- Si es **menor**, se convierte en negro (0).

3. 255

- Es el valor que se asigna a los píxeles **que cumplen** la condición.
- Es decir, los que sean ≥ 100 se volverán **blancos (255)**.

4. cv2.THRESH_BINARY

- Modo de umbralización binaria:
 - Píxeles $\geq$ umbral $\rightarrow$ 255 (blanco)
 - Píxeles $<$ umbral $\rightarrow$ 0 (negro)

3. Función para capturar imágenes y asociarlas a una clase

def capturar_datos(objeto, cantidad, etiqueta, datos, etiquetas):

cap = cv2.VideoCapture(0)

print(f"Captura {cantidad} imágenes del objeto: {objeto}")

capturas = 0

while capturas < cantidad:

ret, frame = cap.read()

cv2.imshow(f"Captura {objeto} (Presiona ESPACIO)", frame)

key = cv2.waitKey(1)

if key == 32: # ESPACIO

feature = extraer_caracteristicas_avanzadas(frame)

datos.append(feature)

etiquetas.append(etiqueta)

capturas += 1

print(f"{objeto} capturado: {capturas} / {cantidad}")

elif key == 27: # ESC

break

```
cap.release()
```

```
cv2.destroyAllWindows()
```

¿Qué hace?

- Muestra la imagen en tiempo real.
- Al presionar **ESPACIO**, extrae características del objeto que aparece en pantalla y lo etiqueta.
- Guarda los vectores de características y sus etiquetas para entrenamiento.

4. Capturar ejemplos de objetos

```
X = []
```

```
y = []
```

```
capturar_datos("taza", 20, "taza", X, y)
```

```
capturar_datos("botella", 20, "botella", X, y)
```

¿Qué hace?

- Captura 20 imágenes de cada objeto.
- Almacena los vectores en X y las etiquetas en y.

Aquí se “**enseña**” al **modelo** cómo es una taza o una botella, a partir de ejemplos reales.

5. Entrenar el modelo KNN

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X, y)
```

```
print("Modelo entrenado correctamente")
```

¿Qué hace?

- Entrena el modelo con los datos capturados.
- `n_neighbors=3` significa que la predicción se hace viendo las 3 muestras más cercanas en el espacio de características.

¿Qué significa `n_neighbors=3` en KNN?

KNN (K-Nearest Neighbors, o "K Vecinos Más Cercanos") es un algoritmo de clasificación supervisada. En lugar de aprender reglas, KNN memoriza los ejemplos que le diste y, cuando ve un nuevo caso, busca los ejemplos más parecidos y vota por la clase que predomina entre ellos.

El número 3 significa:

"Para clasificar un nuevo objeto, voy a buscar los 3 ejemplos más parecidos que tengo, y me voy a quedar con la clase que tenga mayoría entre ellos."

6. Clasificar objetos en tiempo real

```
cap = cv2.VideoCapture(0)
```

```
print("Presiona ESPACIO para predecir el objeto")
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    display = frame.copy()
```

```
    key = cv2.waitKey(1)
```

```
    if key == 32:
```

```
        feature = extraer_caracteristicas_avanzadas(frame).reshape(1, -1)
```

```
        pred = model.predict(feature)[0]
```

```
        print(f" Predicción: {pred}")
```

```
        cv2.putText(display, f"Es una {pred}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX,  
                    1, (0, 255, 0), 2)
```

```
    elif key == 27:
```

```
        break
```

```
cv2.imshow("Clasificador", display)
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

¿Qué hace?

- Muestra la webcam.
- Al presionar **ESPACIO**, clasifica lo que ve basándose en lo que aprendió.
- Escribe en pantalla el resultado de la predicción.